Rockchip DSMC 开发文档

文件标识：RK-KF-YF-C05

发布版本：V1.3.0

日期：2026-05-19

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文为Rockchip DSMC模块的kernel开发提供说明和使用方法。

产品版本

芯片名称	内核版本
RK3576、RK3506、RV1126B	kernel 6.1
RK3572	kernel 6.12

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	何智欢	2024-06-14	初始版本
V1.1.0	何智欢	2025-04-22	增加DSMC Note描述
V1.2.0	何智欢	2025-11-25	增加DSMC dts属性相关描述
V1.3.0	何智欢	2026-05-19	增加DSMC 本地DMA相关描述

目录

Rockchip DSMC 开发文档

1. 名称解释
2. 概述
3. DSMC驱动
   - 3.1 驱动文件
   - 3.2 DTS节点配置
   - 3.3 内核配置
4. 内核态对DSMC从设备内存的访问
   - 4.1 调用驱动接口
   - 4.2 直接访问
5. 用户态对DSMC的访问
   - 5.1 通过特定节点访问
   - 5.2 直接访问
6. DSMC slave内存空间分配
   - 6.1 PSRAM
   - 6.2 LocalBus
7. DSMC LocalBus host与slave的数据交互
   - 7.1 FIFO
   - 7.2 Register
8. 本地 DMA（Local DMA）
   - 8.1 数据结构
   - 8.2 传输方向
   - 8.3 突发类型
   - 8.4 配置宏
   - 8.5 驱动接口
   - 8.6 单节点传输示例
   - 8.7 多节点链表传输示例
   - 8.8 注意事项
9. 注意事项
   - 9.1 DSMC IO 接口频率
   - 9.2 确认DSMC的位宽
   - 9.3 DSMC开启后出现无法进系统的问题
   - 9.4 DSMC自检状态异常

1. 名称解释

- DSMC：Double Data Rate Serial Memory Controller，双倍速率串行存储器控制器
- PSRAM：Pseudo static random access memory，伪静态随机存储器
- DPRAM：Dual Port Random Access Memory，双向随机存取存储器

2. 概述

Double Data Rate Serial Memory Controller（DSMC），双倍速率串行存储器控制器，通过命令、地址、数据线分时复用，数据上下沿传输，具有少引脚数、高带宽的特点。数据线位宽支持x8、x16，最多支持 4 个chip select。传输协议支持HyperBus PSRAM、Xccela PSRAM和 LocalBus。若使用LocalBus协议，从设备需使用RK开发的slave模型，或者传输协议相同。若使用HyperBus PSRAM、Xccela PSRAM协议，从设备支持winbond、AP memory、Cypress、ISSI等厂家生产的PSRAM颗粒。

3. DSMC驱动

3.1 驱动文件

DSMC驱动文件位置：

```
drivers/memory/rockchip/dsmc-host.c          /* 主要驱动程序 */
drivers/memory/rockchip/dsmc-controller.c    /* DSMC控制器行为配置 */
drivers/memory/rockchip/dsmc-lb-device.c     /* DSMC LocalBus设备 */
```

3.2 DTS节点配置

```
    dsmc: dsmc@2a280000 {
        ...
        clock-frequency = <100000000>;          /* DSMC接口频率设置 */
        ...
        /* 从设备属性 */
        slave {
            rockchip,dqs-dll = <0x20 0x20       /* 从设备CS0的DQS0、DQS1 DLL延迟参数 */
                   0x20 0x20                    /* 从设备CS1的DQS0、DQS1 DLL延迟参数 */
                   0x20 0x20                    /* 从设备CS2的DQS0、DQS1 DLL延迟参数 */
                   0x20 0x20>;                  /* 从设备CS3的DQS0、DQS1 DLL延迟参数 */
            /*
```

```
             * rockchip,ranges：DSMC访问从设备内存的基地址，大小；
             * 若不同CS的内存空间大小不同，那么需要配置最大的。
             * rockchip,ranges = <0x0 0x10000000 0x0 0x2000000> 含义：若外设是PSRAM，
             * 那么每个CS都分配0x2000000大小的内存空间；
             * 若外设是LocalBus，那么每个region都分配0x2000000大小的内存空间。
             */
            rockchip,ranges = <0x0 0x10000000 0x0 0x2000000>;
            rockchip,slave-dev = <&dsmc_slave>;
        };
    };

    dsmc_slave: dsmc_slave {
        compatible = "rockchip,dsmc-slave";
        rockchip,clk-mode = <0>;            /* clk 模式，仅限LocalBus */
        status = "disabled";
        /* 从设备是PSRAM（HyperBus PSRAM或Xccela PSRAM）时，开启对应从设备CS的节点 */
        psram {
            /* 使用psram时无需指定位宽，驱动内自适应 */
            psram0 {
                status = "disabled";    /* 若从设备CS0为PSRAM，则改为“okay” */
            };
            psram1 {
                status = "disabled";    /* 若从设备CS1为PSRAM，则改为“okay” */
            };
            psram2 {
                status = "disabled";    /* 若从设备CS2为PSRAM，则改为“okay” */
            };
            psram3 {
                status = "disabled";    /* 若从设备CS3为PSRAM，则改为“okay” */
            };
        };

        /* 从设备是LocalBus设备时，开启、配置对应节点 */
        lb-slave {
            rockchip,io-width = <16>;   /* 指定LocalBus slave的数据位宽，根据slave设计修改 */
            dsmc_lb_slave0: lb-slave0 {
                rockchip,mtr-timing = <1 0 0 0 0 0 2 2>;    /* 核心timing，可根据slave设计修改
*/
                rockchip,int-en = <0x0>;    /* 当前CS使用引脚上的interrupt序号 */
                status = "disabled";    /* 若从设备CS0为LocalBus设备，则改为“okay” */
                dsmc_p0_region: region {
                    dsmc_p0_region0: region0 {  /* 此从设备region0的属性 */
                        rockchip,attribute = "Merged FIFO"; /* region0 为从设备可merge FIFO
*/
                        rockchip,ca-addr-width = <0>;   /* CA传输格式，0：32bit，1：16bit */
                        rockchip,dummy-clk-num = <1>;   /* 写传输结束后额外输出clk的数量 */
                        rockchip,cs0-be-ctrled = <0>;   /* 从设备CS0被从设备CS1、2、3控制 */
                        rockchip,cs0-ctrl = <0>;    /* 从设备CS0控制从设备CS1、2、3 */
                        status = "disabled";
                    };
                    dsmc_p0_region1: region1 {  /* 此从设备region1的属性 */
                        rockchip,attribute = "No-Merge FIFO";   /* region1 为从设备不可merge
FIFO */
```

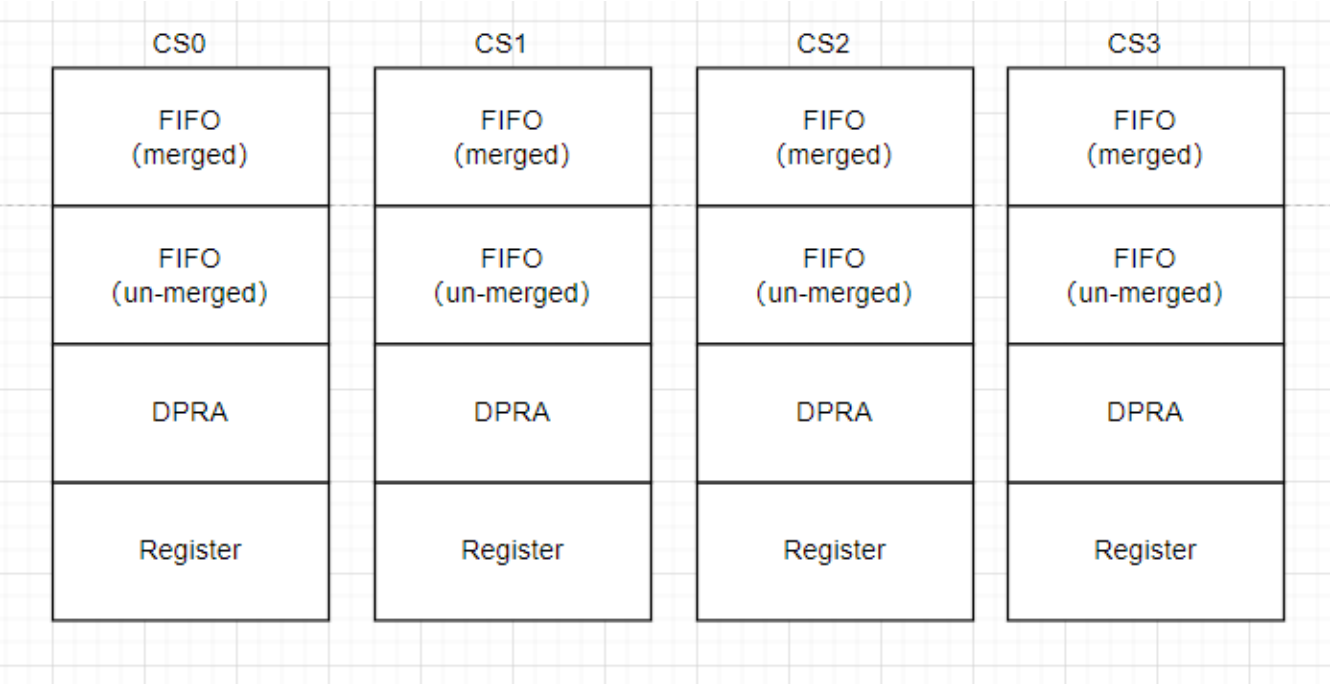
```
                        rockchip,ca-addr-width = <0>;
                        rockchip,dummy-clk-num = <1>;
                        rockchip,cs0-be-ctrled = <0>;
                        rockchip,cs0-ctrl = <0>;
                        status = "disabled";
                    };
                    dsmc_p0_region2: region2 {  /* 此从设备region2的属性 */
                        rockchip,attribute = "DPRA";    /* region2 为从设备DPRAM */
                        rockchip,ca-addr-width = <0>;
                        rockchip,dummy-clk-num = <1>;
                        rockchip,cs0-be-ctrled = <0>;
                        rockchip,cs0-ctrl = <0>;
                        status = "disabled";
                    };
                    dsmc_p0_region3: region3 {  /* 此从设备region3的属性 */
                        rockchip,attribute = "Register";    /* region3 为从设备寄存器 */
                        rockchip,ca-addr-width = <0>;
                        rockchip,dummy-clk-num = <1>;
                        rockchip,cs0-be-ctrled = <0>;
                        rockchip,cs0-ctrl = <0>;
                        status = "disabled";
                    };
                };
            };
            dsmc_lb_slave1: lb-slave1 {
                rockchip,mtr-timing = <1 0 0 0 0 0 2 2>;    /* 核心timing，可根据slave设计修改
*/
                rockchip,int-en = <0x1>;    /* 当前CS使用引脚上的interrupt序号 */
                status = "disabled";    /* 若从设备CS1为LocalBus设备，则改为“okay” */
                dsmc_p1_region: region {
                    dsmc_p1_region0: region0 {
                        rockchip,attribute = "Merged FIFO";
                        rockchip,ca-addr-width = <0>;
                        rockchip,dummy-clk-num = <1>;
                        rockchip,cs0-be-ctrled = <0>;
                        rockchip,cs0-ctrl = <0>;
                        status = "disabled";
                    };
                    dsmc_p1_region1: region1 {
                        rockchip,attribute = "No-Merge FIFO";
                        rockchip,ca-addr-width = <0>;
                        rockchip,dummy-clk-num = <1>;
                        rockchip,cs0-be-ctrled = <0>;
                        rockchip,cs0-ctrl = <0>;
                        status = "disabled";
                    };
                    dsmc_p1_region2: region2 {
                        rockchip,attribute = "DPRA";
                        rockchip,ca-addr-width = <0>;
                        rockchip,dummy-clk-num = <1>;
                        rockchip,cs0-be-ctrled = <0>;
                        rockchip,cs0-ctrl = <0>;
                        status = "disabled";
```

```
                };
                dsmc_p1_region3: region3 {
                    rockchip,attribute = "Register";
                    rockchip,ca-addr-width = <0>;
                    rockchip,dummy-clk-num = <1>;
                    rockchip,cs0-be-ctrled = <0>;
                    rockchip,cs0-ctrl = <0>;
                    status = "disabled";
                };
            };
        };
        dsmc_lb_slave2: lb-slave2 {
            rockchip,mtr-timing = <1 0 0 0 0 0 2 2>;    /* 核心timing，可根据slave设计修改
*/
            rockchip,int-en = <0x2>;    /* 当前CS使用引脚上的interrupt序号 */
            status = "disabled";    /* 若从设备CS2为LocalBus设备，则改为“okay” */
            dsmc_p2_region: region {
                dsmc_p2_region0: region0 {
                    rockchip,attribute = "Merged FIFO";
                    rockchip,ca-addr-width = <0>;
                    rockchip,dummy-clk-num = <1>;
                    rockchip,cs0-be-ctrled = <0>;
                    rockchip,cs0-ctrl = <0>;
                    status = "disabled";
                };
                dsmc_p2_region1: region1 {
                    rockchip,attribute = "No-Merge FIFO";
                    rockchip,ca-addr-width = <0>;
                    rockchip,dummy-clk-num = <1>;
                    rockchip,cs0-be-ctrled = <0>;
                    rockchip,cs0-ctrl = <0>;
                    status = "disabled";
                };
                dsmc_p2_region2: region2 {
                    rockchip,attribute = "DPRA";
                    rockchip,ca-addr-width = <0>;
                    rockchip,dummy-clk-num = <1>;
                    rockchip,cs0-be-ctrled = <0>;
                    rockchip,cs0-ctrl = <0>;
                    status = "disabled";
                };
                dsmc_p2_region3: region3 {
                    rockchip,attribute = "Register";
                    rockchip,ca-addr-width = <0>;
                    rockchip,dummy-clk-num = <1>;
                    rockchip,cs0-be-ctrled = <0>;
                    rockchip,cs0-ctrl = <0>;
                    status = "disabled";
                };
            };
        };
        dsmc_lb_slave3: lb-slave3 {
```

```
                rockchip,mtr-timing = <1 0 0 0 0 0 2 2>;    /* 核心timing，可根据slave设计修改
*/
                rockchip,int-en = <0x3>;    /* 当前CS使用引脚上的interrupt序号 */
                status = "disabled";    /* 若从设备CS3为LocalBus设备，则改为“okay” */
                dsmc_p3_region: region {
                    dsmc_p3_region0: region0 {
                        rockchip,attribute = "Merged FIFO";
                        rockchip,ca-addr-width = <0>;
                        rockchip,dummy-clk-num = <1>;
                        rockchip,cs0-be-ctrled = <0>;
                        rockchip,cs0-ctrl = <0>;
                        status = "disabled";
                    };
                    dsmc_p3_region1: region1 {
                        rockchip,attribute = "No-Merge FIFO";
                        rockchip,ca-addr-width = <0>;
                        rockchip,dummy-clk-num = <1>;
                        rockchip,cs0-be-ctrled = <0>;
                        rockchip,cs0-ctrl = <0>;
                        status = "disabled";
                    };
                    dsmc_p3_region2: region2 {
                        rockchip,attribute = "DPRA";
                        rockchip,ca-addr-width = <0>;
                        rockchip,dummy-clk-num = <1>;
                        rockchip,cs0-be-ctrled = <0>;
                        rockchip,cs0-ctrl = <0>;
                        status = "disabled";
                    };
                    dsmc_p3_region3: region3 {
                        rockchip,attribute = "Register";
                        rockchip,ca-addr-width = <0>;
                        rockchip,dummy-clk-num = <1>;
                        rockchip,cs0-be-ctrled = <0>;
                        rockchip,cs0-ctrl = <0>;
                        status = "disabled";
                    };
                };
            };
        };
    };
```

用户需根据实际从设备的类型，开启对应的节点。如PSRAM设备，需根据PCB外接CS的序号，开启对应psramx节点。具体是HyperBus还是Xccela PSRAM，由驱动自动识别。当从设备是RK设计 DSMC slave时，根据PCB外接CS的序号，开启对应的 lb-slavex 节点。另外还需根据从设备的属性修改对应region的配置。其中 `rockchip,clk-mode` 控制支持的三种clk行为： `rockchip,clk-mode = 0` 即 CS 拉高期间无时钟，拉低期间有时钟； `rockchip,clk-mode = 1` 即无论 CS 怎么变化，时钟一直有，slave 可将其作为参考时钟，但是此模式下无法跑高频且各类 AC timing 可调参数不可用； `rockchip,clk-mode = 2` 即 CS拉高和拉低期间都有时钟，但是CS跳变沿前后会关闭几个时钟。

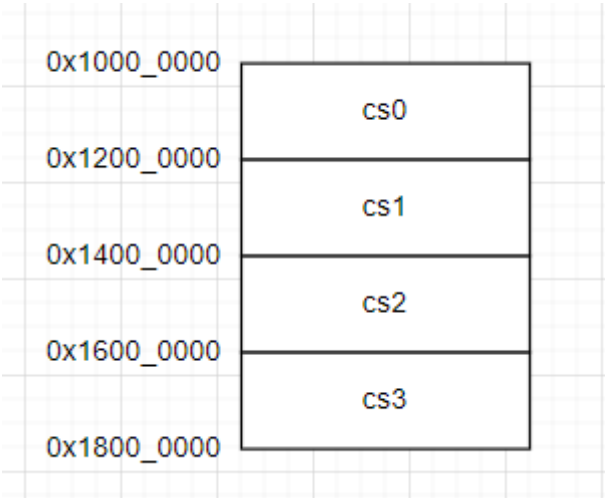
作为 LocalBus 设备时，设备空间示意图如下：



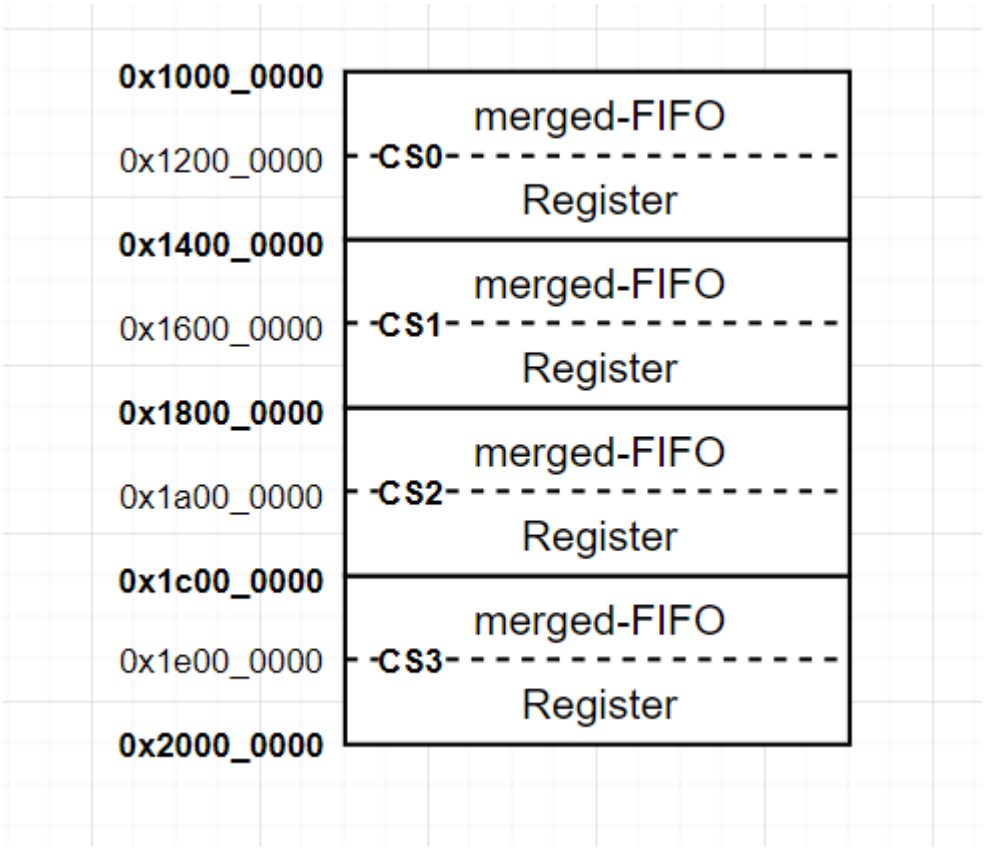
每个从设备片选CS的访问空间都可以分成1、2、4个region（均分），只需要在DTS开启对应属性region的status。

对于`rockchip,ranges = <0x0 0x10000000 0x0 0x2000000>;`属性，配置的是从设备内存空间的起始地址和大小，不同外设类型有不同含义。

若外设是PSRAM，`rockchip,ranges`配置的是最大CS空间的大小。每个CS的内存空间划分如下：



若外设是 LocalBus，`rockchip,ranges`配置的是最大region空间的大小。在只开启Register和merged-FIFO 2个region的情况下，每个CS的region空间划分如下：



3.3 内核配置

```
Symbol: ROCKCHIP_DSMC [=y]
Type  : bool
Prompt: Rockchip DSMC(Double Data Rate Serial Memory Controller) driver
Depends on: MEMORY [=y]
Location:
  -> Device Drivers
    -> Memory Controller drivers (MEMORY [=y])
      -> Rockchip DSMC(Double Data Rate Serial Memory Controller) driver (ROCKCHIP_DSMC
[=y])
```

4. 内核态对DSMC从设备内存的访问

4.1 调用驱动接口

在DSMC驱动里`drivers/memory/rockchip/dsmc-host.c`实现了如下几种访问接口：

```
const char *rockchip_dsmc_get_compat(int index);
struct rockchip_dsmc_device *rockchip_dsmc_find_device_by_compat(const char *compat);

static struct dsmc_ops rockchip_dsmc_ops = {
    .read = dsmc_read,
    .write = dsmc_write,
    .copy_from = dsmc_copy_from,
    .copy_from_state = dsmc_copy_from_state,
    .copy_to = dsmc_copy_to,
    .copy_to_state = dsmc_copy_to_state,
    .copy_sg = dsmc_copy_sg,
    .copy_sg_state = dsmc_copy_sg_state,
};
```

通过调用函数`rockchip_dsmc_find_device_by_compat()`查找dsmc设备，并获取这个设备的私有参数。并通过`dsmc_dev.ops`实现CPU，DMA 对DSMC从设备的访问。其中`ops->read`、`ops->write`为CPU读写DSMC从设备的内存空间。`ops->copy_from`用于DMA读取从设备内存，并写入host端内存。`ops->copy_to`用于DMA从host端内存写入从设备内存。对于支持私有DMA（本地DMA）的DSMC版本，`ops->copy_from`、`ops->copy_to`会优先使用私有DMA。

注意：DSMC 控制器版本 4.0 及以上（如 RK3572）支持本地 DMA 功能。`ops->copy_sg`接口专门用于本地 DMA 的链表传输功能，支持分散-聚集（Scatter-Gather）DMA 传输。详细说明请参考“本地 DMA（Local DMA）”章节。

```
static int test(void)
{
    u32 cs;
    struct rockchip_dsmc_device *dsmc_dev;

    dsmc_dev = rockchip_dsmc_find_device_by_compat(rockchip_dsmc_get_compat(0));
    if (dsmc_dev == NULL){
        printk("error: can not find dsmc device\n");
        return -1;
    }
    for (cs = 0; cs < DSMC_MAX_SLAVE_NUM; cs++) {
        if (dsmc_dev->dsmc.cfg.cs_cfg[cs].device_type == DSMC_UNKNOWN_DEVICE)
            continue;
        dsmc_dev->ops->write(dsmc_dev, cs, 0, test_addr, test_data);
        /* TODO */
    }

    return 0;
}
```

4.2 直接访问

CPU或master可直接访问DSMC slave memory空间（支持Byte，half-word，word的随机地址访问；支持cacheable、uncacheable、write combine映射方式）。

5. 用户态对DSMC的访问

5.1 通过特定节点访问

在LocalBus使能的情况下，DSMC驱动会在`/dev/dsmc/`创建4个从设备片选CS，每个CS下会创建4个 memory region，各属性分别对应DTS的描述。若需要访问 merged-FIFO，对应为region0，需操作对应片选CS下的对应region节点。具体如下：

1. 使用 open() 接口 打开对应region设备节点，获取文件描述符；
2. 使用 mmap() 系统调用将上述region设备内存映射到进程地址空间，获取映射地址；
3. 读写设备内存；
4. 使用 munmap() 解除内存映射；
5. 使用 close() 关闭文件描述符。

代码示例如下：

```
    device_name = "/dev/dsmc/cs0/region0";
    wantbytes = 0x200000;
    memfd = open(device_name, O_RDWR | O_SYNC);
    if (memfd == -1) {
        fprintf(stderr, "failed to open %s for physical memory: %s\n",
            device_name, strerror(errno));
        exit(EXIT_FAILURE);
    }
    /* 使用 O_SYNC 标志打开文件和 MAP_LOCKED 标志进行 mmap 操作的memory空间是 uncached 的 */
    buf = (void volatile *) mmap(0, wantbytes, PROT_READ | PROT_WRITE,
                                 MAP_SHARED | MAP_LOCKED, memfd,
                                 0x0);
    if (buf == MAP_FAILED) {
        fprintf(stderr, "failed to mmap %s for physical memory: %s\n",
                device_name, strerror(errno));
        exit(EXIT_FAILURE);
    }
    bufsize = wantbytes;
    halflen = bufsize / 2;
    count = halflen / sizeof(u32);
    bufa = (u32v *) buf;
    bufb = (u32v *) ((size_t) buf + halflen);

    test_dsmc(bufa, bufb, count);/* 读写DSMC slave内存 */

    if (munmap((void*)buf, wantbytes) == -1) {
        perror("munmap");
        close(memfd);
        exit(EXIT_FAILURE);
    }

    close(memfd);
```

5.2 直接访问

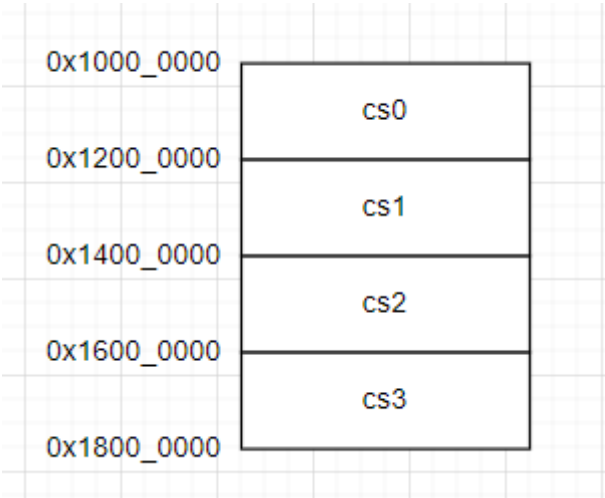
CPU可以通过 /dev/mem 映射的DSMC slave memory空间直接进行访问，如RK3576上使用 io 命令进行DSMC slave memory 的 region0 访问：io -4 0x10000000。

6. DSMC slave内存空间分配

DSMC slave的空间分配由`rockchip,ranges`属性控制，配置的是从设备内存空间的起始地址和大小。

6.1 PSRAM

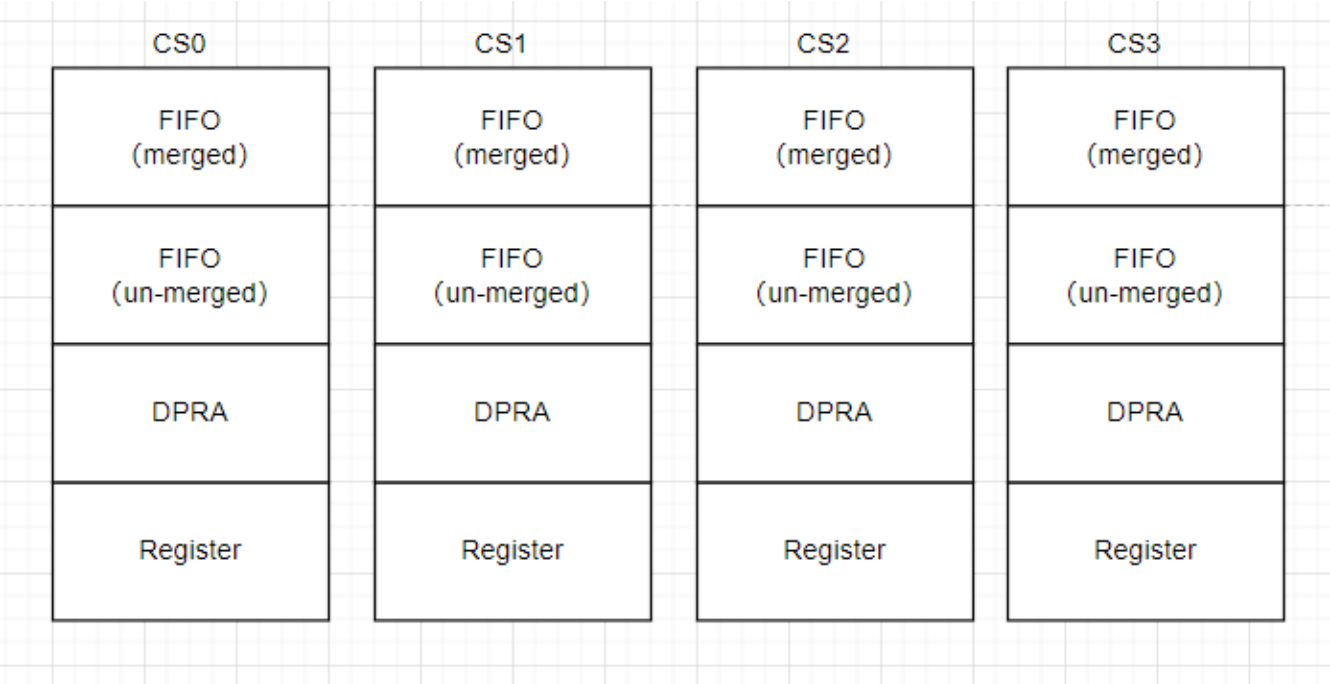
例如配置`rockchip,ranges = <0x0 0x10000000 0x0 0x2000000>;`，若外设是PSRAM，`rockchip,ranges`配置的是最大CS空间的大小。对DSMC控制器来说，每个片选CS的容量是相同的。若实际不同片选CS贴不同容量PSRAM，用户访问时应当注意边界。每个CS的内存空间划分如下：



注意：DSMC理论上支持不同片选CS是不同厂家PSRAM，支持容量不同，但不同片选CS的位宽（x8 或 x16）必须相同。

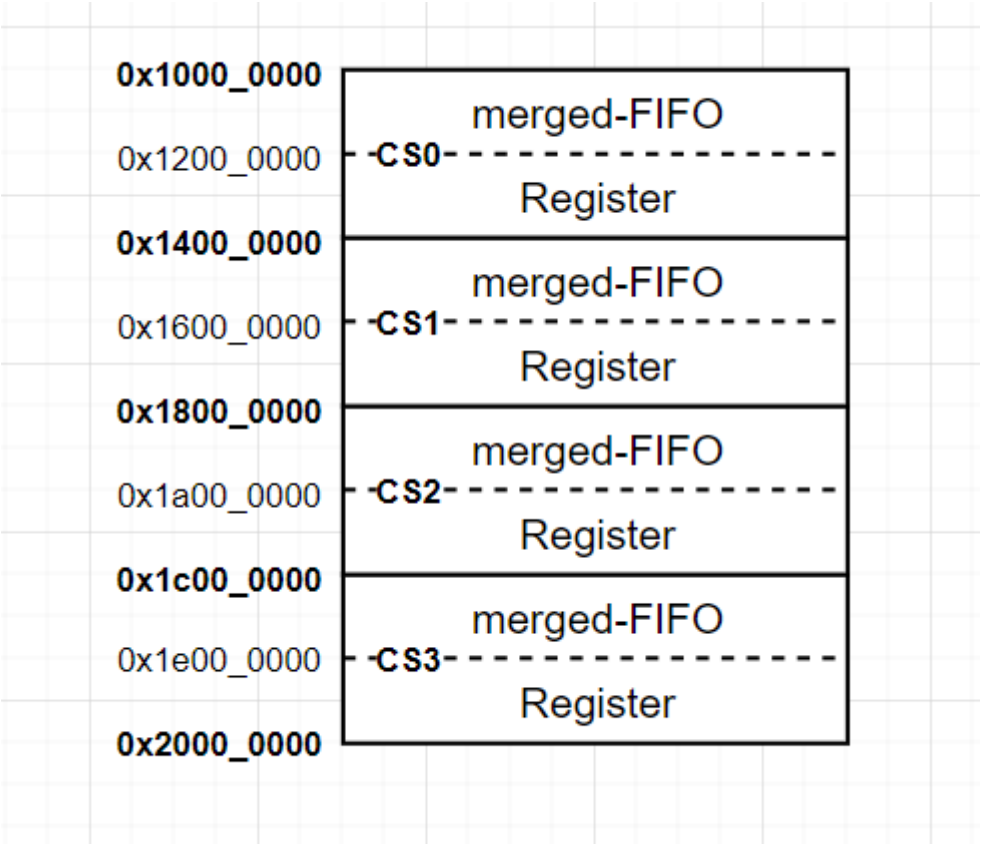
6.2 LocalBus

若外设是 LocalBus，对于DSMC控制器来说，每个片选CS的容量也是相同的。每个片选CS都可以均分成1、2、4个region，每个region的属性可以是DPRAM、Register、merged FIFO和un-merged FIFO。若实际每个CS的region容量不同，用户访问时应当注意边界。



对于LocalBus `rockchip,ranges` 配置的是最大region空间的大小。

若一个片选CS开启2个region，每个region大小由DTS（DSMC节点slave设备的`rockchip,ranges = 0x0 0x10000000 0x0 0x2000000`）决定。各个CS的各region的内存空间分配如下：



7. DSMC LocalBus host与slave的数据交互

7.1 FIFO

当DSMC使用LocalBus 协议，且外接RK slave时，slave端有一段FIFO，当访问的region属性是merged-FIFO或者un-merged-FIFO时，host端传输的数据经过FIFO后，slave端会再次写入slave端的内存，如DDR，SRAM等。这段FIFO不可见，对于DSMC host端来说，slave端的DDR，SRAM内存空间即是slave端的内存空间。通过DSMC写入的数据，最终都将被写入slave端的内存，在使用时应注意slave端内存空间的管理和数据一致性。

7.2 Register

当DSMC使用LocalBus 协议，且外接RK slave时，有一段SLAVE_CSR Register，当访问的region属性是Register时，即是访问这段SLAVE_CSR Register。这段寄存器可用于host与slave的信息快速传递。

可用于信息交互的寄存器：

APP_CON0	LBC_CON0
APP_CON1	LBC_CON1
...	...
APP_CON14	LBC_CON14
APP_CON15	LBC_CON15

其中 APP_CONx 寄存器，slave有读写权限，host端只有读权限；LBC_CONx寄存器 slave只有读权限，host端有读写权限。

- 当host写入LBC_CONx寄存器，会触发host2slave中断给slave端CPU，用于处理host端传入的数据。
- 当slave写入APP_CONx寄存器后，通过IO引脚 INT 触发slave2host中断，传入host 端，host端CPU也可以响应中断，获取slave传递过来的数据。DSMC 接收 INT 信号后，也可以发起DMA 硬件请求，DMA开始搬移数据（DMA需提前配置好）。

典型场景1

host端将信息写入LBC_CONx寄存器，将触发host2slave中断，slave端从LBC_CONx寄存器读取信息；slave端将信息写入 APP_CONx 寄存器，将触发slave2host中断，host端从APP_CONx寄存器读取信息。

典型场景2

host端DSMC、DMA配置好后，host端通过写LBC_CONx寄存器通知slave端，然后slave端通过写APP_CONx，从INT引脚返回host一个有效信号，DSMC host接收后，发起DMA 硬件请求，开启DMA搬移。

注意：APP_CON15、LBC_CON15 已被使用。DSMC 使用DMA硬件请求的驱动已实现，当host端DMA配置完成，host端将LBC_CON15 原值+1 写入LBC_CON15 寄存器，slave接收到数据后，将APP_CON15 写1 触发slave2host中断，DSMC host接收后自动发起一定数量的DMA 硬件请求，触发DMA搬移。

8. 本地 DMA（Local DMA)

从 DSMC 控制器版本 4.0 开始，新增了本地 DMA（Local DMA）功能。本地 DMA 是 DSMC 控制器内置的 DMA 引擎，可实现 AHB 总线与 LocalBus 之间的数据传输，无需依赖系统公共 DMA 控制器。

版本支持：

DSMC 版本	本地 DMA 支持	芯片平台
< 4.0	不支持	RK3576, RV1126B, RK3506
>= 4.0	支持	RK3572

主要特性：

- **独立 DMA 引擎**：不占用系统公共 DMA 资源
- **链表传输支持**：支持分散-聚集（Scatter-Gather）DMA 传输
- **多种突发模式**：支持 Single、INCR4、INCR8、INCR16 四种突发类型
- **双向传输**：支持 AHB→LBC（写入从设备）和 LBC→AHB（读取从设备）
- **硬件触发模式**：支持通过外部中断引脚触发 DMA 传输

8.1 数据结构

本地 DMA 链表节点：

```
/**
 * struct dsmc_local_dma_link_list - DSMC 本地 DMA 链表节点
 *
 * 硬件支持的分散-聚集 DMA 链表节点。每个节点描述一个 DMA 传输段。
 *
 * @ll_ctl: 链表控制寄存器完整值
 * @ll_rdwr: 传输方向: 0=DSMC 写(AHB->LBC), 1=DSMC 读(LBC->AHB)
 * @ll_burst: 突发类型: 0=Single, 3=INCR4, 5=INCR8, 7=INCR16
 * @p_trans_en: 外设传输使能: 1=使用外设请求模式(外部中断引脚触发)
 * @ll_trans_len: 传输长度，单位 8 字节
 * @is_ll_last: 链表结束标志: 1=链表中最后一个节点
 * @ll_ahb_addr: AHB 地址寄存器完整值（40 位）
 * @ll_lbc_addr: LocalBus 地址寄存器完整值（32 位）
 * @next: 下一个链表节点的物理地址
 */
struct dsmc_local_dma_link_list {
    union {
        uint64_t ll_ctl;
        struct {
            uint64_t ll_rdwr:1;
            uint64_t ll_burst:3;
            uint64_t p_trans_en:1;
            uint64_t ll_trans_len:11;
            uint64_t is_ll_last:1;
            uint64_t reserved:47;
        } ll_ctl_f;
    };
    union {
```

```
        uint64_t ll_ahb_addr;
        struct {
            uint64_t ahb_addr:40;
            uint64_t reserved:24;
        } ll_ahb_addr_f;
    };
    union {
        uint64_t ll_lbc_addr;
        struct {
            uint64_t lbc_addr:32;
            uint64_t reserved:32;
        } ll_lbc_addr_f;
    };
    uint64_t next;
};
```

8.2 传输方向

```
enum dsmc_local_dma_transfer_direction {
    LOCAL_DMA_WRITE_TO_LBC = 0,     /* AHB -> LBC (写入从设备) */
    LOCAL_DMA_READ_FROM_LBC = 1,    /* LBC -> AHB (读取从设备) */
};
```

8.3 突发类型

```
enum dsmc_local_dma_burst_type {
        LOCAL_DMA_BURST_SINGLE = 0,             /* 单次传输 */
        LOCAL_DMA_BURST_INCR4  = 3,             /* 4 次突发传输 */
        LOCAL_DMA_BURST_INCR8  = 5,             /* 8 次突发传输 */
        LOCAL_DMA_BURST_INCR16 = 7,             /* 16 次突发传输 */
};
```

8.4 配置宏

```
/* DMA传输字节到传输长度的转换 */
#define LOCAL_DMA_BYTES_TO_TRANS_LEN(size)

/* 写入节点（使用默认 INCR16 突发） */
#define LOCAL_DMA_WRITE_NODE(node, ahb_addr, lbc_addr, size, is_last)

/* 读取节点（使用默认 INCR16 突发） */
#define LOCAL_DMA_READ_NODE(node, lbc_addr, ahb_addr, size, is_last)

/* 写入节点（自定义突发类型） */
#define LOCAL_DMA_WRITE_NODE_BURST(node, ahb_addr, lbc_addr, size, burst, is_last)
```

```
/* 读取节点（自定义突发类型） */
#define LOCAL_DMA_READ_NODE_BURST(node, lbc_addr, ahb_addr, size, burst, is_last)

/* 链接节点到下一个物理地址 */
int dsmc_local_dma_node_link_to_phys(struct dsmc_local_dma_link_list *node,
                                                                    phys_addr_t
next_phys);

/* 设置节点为链表最后一个节点 */
int dsmc_local_dma_node_set_last(struct dsmc_local_dma_link_list *node);
```

8.5 驱动接口

```
/* 本地 DMA 链表传输接口 */
int (*copy_sg)(struct rockchip_dsmc_device *dsmc_dev,
                           struct dsmc_local_dma_link_list *sg_list,
                           phys_addr_t sg_list_phys, uint32_t cs);

/* 状态查询接口 */
int (*copy_sg_state)(struct rockchip_dsmc_device *dsmc_dev);
```

8.6 单节点传输示例

```
#include <asm/cacheflush.h>
#include "dsmc-host.h"

struct dsmc_local_dma_link_list *sg_list;
phys_addr_t sg_list_phys;
void *src_buf;
dma_addr_t src_phys;
u32 size = 0x100;

/* 1. 分配链表节点内存 */
sg_list = dma_alloc_coherent(dev, sizeof(struct dsmc_local_dma_link_list),
                                                        &sg_list_phys, GFP_KERNEL);

/* 2. 配置写入节点（AHB -> LBC） */
LOCAL_DMA_WRITE_NODE(sg_list, src_phys, dsmc_map->phys, size, 1);

/* 3. 同步链表节点到内存 */
#ifdef CONFIG_ARM64
dcache_clean_inval_poc((unsigned long)sg_list,
                                          (unsigned long)sg_list + sizeof(*sg_list));
#else
flush_cache_all();
#endif
```

```
/* 4. 启动 DMA 传输 */
ops->copy_sg(dsmc_dev, sg_list, sg_list_phys, cs);

/* 5. 等待传输完成 */
while (ops->copy_sg_state(dsmc_dev)) {
        mdelay(1);
}
```

8.7 多节点链表传输示例

```
#include <asm/cacheflush.h>
#include "dsmc-host.h"

static int multi_node_transfer(struct rockchip_dsmc_device *dsmc_dev, u32 cs)
{
        struct device *dev = dsmc_dev->dsmc.dev;
        const struct dsmc_ops *ops = dsmc_dev->ops;
        struct dsmc_map *map = &dsmc_dev->dsmc.cs_map[cs].region_map[0];        /* 使用固定
region 0或者对应启用的region */

        struct dsmc_local_dma_link_list *sg_list[4];
        phys_addr_t sg_list_phys[4];
        void *src_buf[4];
        dma_addr_t src_phys[4];
        u32 size = 0x100;
        int i, ret;

        /* 1. 分配所有链表节点和缓冲区 */
        for (i = 0; i < 4; i++) {
                sg_list[i] = dma_alloc_coherent(dev, sizeof(*sg_list[i]),

&sg_list_phys[i], GFP_KERNEL);
                src_buf[i] = dma_alloc_coherent(dev, size, &src_phys[i], GFP_KERNEL);
        }

        /* 2. 配置链表节点 */
        for (i = 0; i < 4; i++) {
                LOCAL_DMA_WRITE_NODE(sg_list[i], src_phys[i],
                                                        map->phys + size * i, size, 0);

                /* 链接到下一个节点 */
                if (i < 3) {
                        dsmc_local_dma_node_link_to_phys(sg_list[i], sg_list_phys[i + 1]);
                } else {
                        dsmc_local_dma_node_set_last(sg_list[i]);
                }
                #ifdef CONFIG_ARM64
                dcache_clean_inval_poc((unsigned long)sg_list[i],
                                                          (unsigned long)sg_list[i] +
sizeof(*sg_list[i]));
                #else
```

```
                flush_cache_all();
                #endif
        }

        /* 3. 启动 DMA 传输 */
        ret = ops->copy_sg(dsmc_dev, sg_list[0], sg_list_phys[0], cs);

        /* 4. 等待完成 */
        while (ops->copy_sg_state(dsmc_dev)) {
                mdelay(1);
        }

        return 0;
}
```

8.8 注意事项

内存对齐：

- 本地 DMA 传输长度单位为 **8 字节**。
- 使用 `LOCAL_DMA_BYTES_TO_TRANS_LEN(size)` 宏进行字节到传输长度的转换。
- 实际传输字节数会向上对齐到 8 字节的整数倍。
- 单次最大传输长度为 2047 个传输单位（16376 字节）。

9. 注意事项

9.1 DSMC IO 接口频率

1. 由dts中`dsmc`节点的参数`clock-frequency = <100000000>;`设置，最终不一定完全可以设置到目标频率，具体偏差取决于当前平台的PLL或晶振输出的频率，最终频率是会按最接近设置频率去配置的。
2. DSMC控制器内部存在**固定的2分频**，所以PLL或晶振提供给DSMC 控制器的频率是DSMC IO接口上频率的2倍。

9.2 确认DSMC的位宽

1. 外接**DSMC slave**位宽无法自动探测，代码默认是x16位宽的。若外接RK3506做DSMC slave，因为此slave只支持 x8 位宽，所以位宽需要修改正确。

   新版本可从dts上修改slave位宽。

```
&dsmc_slave {
        status = "okay";
        lb-slave {
                rockchip,io-width = <8>;
                ...
        };
};
```

旧版本需从驱动代码里修改。

```
diff --git a/drivers/memory/rockchip/dsmc-host.c b/drivers/memory/rockchip/dsmc-host.c
index 44182923acc9..5214c9128320 100644
--- a/drivers/memory/rockchip/dsmc-host.c
+++ b/drivers/memory/rockchip/dsmc-host.c
@@ -782,7 +782,7 @@ static void dsmc_data_init(struct rockchip_dsmc *dsmc)
                                                cs_cfg->rd_bdr_xfer_en = 1;
                                                cs_cfg->wr_bdr_xfer_en = 1;
                                } else {
-                                               cs_cfg->io_width = MCR_IOWIDTH_X16;
+                                               cs_cfg->io_width = MCR_IOWIDTH_X8;
                                                cs_cfg->wrap_size =
DSMC_BURST_WRAPSIZE_16CLK;
                                                cs_cfg->wrap2incr_en = 1;
                                                cs_cfg->acs = 1;
```

2. 若外接**PSRAM**，位宽是自动探测的，不需要手动修改。若外接的PSRAM兼容x8和x16模式，默认使用x16模式。

9.3 DSMC开启后出现无法进系统的问题

因DSMC开启后，会读写访问 slave，若访问出现异常，如read DQS没有返回，将导致DSMC往总线报error，系统接收后进异常，典型log如下。

```
...
[    1.391589] dsmc 21ca0000.dsmc: init cs0 LB device # DSMC驱动加载后会访问slave
[    1.394238] 8<--- cut here ---
[    1.394243] Unhandled fault: external abort on non-linefetch (0x1008) at 0xf0600100
[    1.394250] [f0600100] *pgd=40b0f811, *pte=10000653, *ppte=10000453
[    1.394267] Internal error: Oops - BUG: 1008 [#1] PREEMPT SMP THUMB2
...
```

出现这种问题请**排查DSMC master与slave的硬件连接性**。

应当注意，在DSMC master初始化的时候，**slave必须已经完成上电并初始化**，否则也会出现访问异常的情况。

9.4 DSMC自检状态异常

DSMC驱动在初始化结束之后会对DSMC状态寄存器进行检查，确认初始化是否存在异常。当出现如下异常打印，则说明初始化存在异常。

```
dsmc ff8b0000.dsmc: DSMC: csr status error(0x800)!# 具体状态异常值可能不同
dsmc ff8b0000.dsmc: DSMC status error, please check hardware matched(io, slave etc.)
```

出现这种问题请**排查DSMC master与slave的硬件连接性**。

应当注意，在DSMC master初始化的时候，**slave必须已经完成上电并初始化**，否则也会出现访问异常的情况。